

Tempo logico nei sistemi distribuiti

Happened-before • Lamport Clock • Vector Clock • Mutex

Roberto Marino, PhD

Università di Messina — CdL in Informatica — Reti e Sistemi Distribuiti Mod.B

14 maggio 2026

Perché il tempo è difficile nei sistemi distribuiti

In un sistema centralizzato, ordinare gli eventi è banale: c'è un unico processore, un unico clock, un unico stack di chiamate. Nei sistemi distribuiti questo privilegio non esiste.

Ogni nodo ha il proprio orologio hardware, soggetto a tre fenomeni indipendenti:

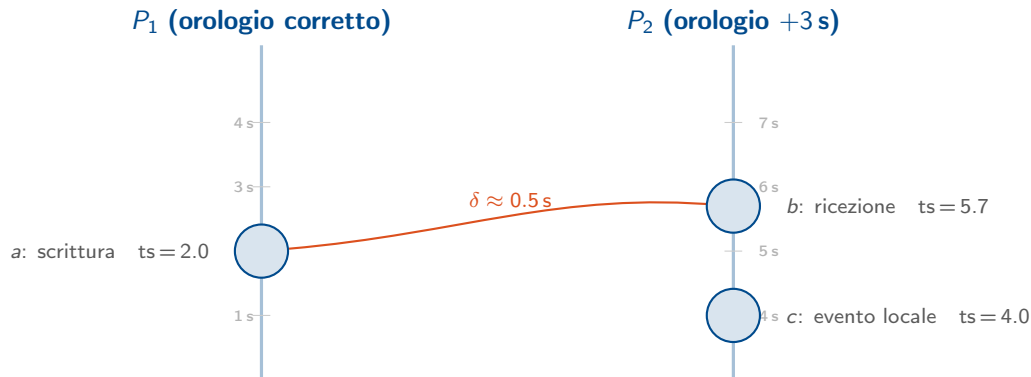
- ▶ **Clock skew** — offset costante tra due orologi: il nodo *A* legge 10:00:00 mentre *B* legge 10:00:03 nello *stesso istante fisico*.
- ▶ **Clock drift** — l'offset cambia nel tempo; i cristalli oscillano a frequenze leggermente diverse, producendo divergenza cumulativa e imprevedibile.
- ▶ **Latenza di rete** — un messaggio inviato al tempo t arriva al tempo $t + \delta$, con δ variabile e ignoto al mittente.

Effetto combinato

Un evento su *B* con timestamp 10:00:02 può essere *fisicamente precedente* a un evento su *A* con timestamp 10:00:01, se l'orologio di *B* è avanti di 3 secondi. Il confronto diretto di timestamp fisici **non** riflette la causalità.

Un esempio concreto di anomalia

Il seguente scenario mostra perché i timestamp fisici ingannano.



Il paradosso nasce perché i timestamp fisici non codificano la *causalità*: il fatto che *b* sia una conseguenza di *a* viene perso. Dobbiamo costruire un *orologio logico* che la rispetti.

Cosa vogliamo davvero: la causalità

Osservare che a è avvenuto prima di b fisicamente non ci dice nulla di utile se non sappiamo se b dipende da a . Ciò che conta è la *relazione causale*: b **può essere influenzato** da a se e solo se a è accaduto prima e le informazioni di a hanno potuto raggiungere b .

Tre situazioni esauriscono tutti i casi possibili:

- ▶ **Stessa macchina, stesso flusso**: se a precede b nello stesso processo, a può aver causato b .
- ▶ **Comunicazione**: se a è l'invio di un messaggio e b la sua ricezione, il contenuto di a è noto a b per definizione.
- ▶ **Transitività**: se a può aver causato b , e b può aver causato c , allora a può aver causato c .

Tutto il resto — eventi su nodi diversi senza messaggi diretti né indiretti — è **concorrente**: non c'è modo di stabilire quale sia avvenuto “prima” in senso causale.

Questa intuizione è stata formalizzata da Leslie Lamport nel 1978 con la relazione *happened-before*.

La relazione *happened-before* ($\rightarrow$)

Definizione (Lamport, 1978)

$\rightarrow$ è la *più piccola* relazione sugli eventi che soddisfa contemporaneamente le tre regole seguenti.

1. **Ordine locale.** Se a e b appartengono allo stesso processo e a precede b nell'esecuzione, allora $a \rightarrow b$.
2. **Passaggio di messaggi.** Se a è l'invio di un messaggio e b è la sua ricezione, allora $a \rightarrow b$.
3. **Transitività.** Se $a \rightarrow b$ e $b \rightarrow c$, allora $a \rightarrow c$.

La relazione è *irriflessiva* ($a \not\rightarrow a$) e *aciclica* (nessun ciclo causale, altrimenti avremmo viaggi nel tempo). Non è totale: eventi senza percorso causale tra loro sono **concorrenti** ($a \parallel b$, ovvero $a \not\rightarrow b$ e $b \not\rightarrow a$).

Nota importante

$a \rightarrow b$ non significa che a ha causato b , ma solo che *avrebbe potuto*: è una sovrastima conservativa della causalità. È l'unica garanzia ottenibile senza accesso a un orologio globale perfetto.

L'idea: un contatore che “segue” la causalità

Vogliamo assegnare ad ogni evento un numero intero $C()$ tale che, se $a \rightarrow b$, allora $C(a) < C(b)$. Non servono orologi fisici sincronizzati: basta un contatore locale che ogni processo aggiorna secondo tre regole semplici.

L'idea chiave è che **il clock deve avanzare ad ogni evento**, e che **un messaggio porta con sé il valore del mittente** affinché il destinatario non possa “restare indietro”.

Ogni processo P_i mantiene un contatore c_i , inizialmente 0.

R1 — Evento interno

Prima di ogni evento locale:

$$c_i \leftarrow c_i + 1$$

R2 — Invio messaggio

Incrementa, poi allega il valore:

$$c_i \leftarrow c_i + 1, \quad m.ts \leftarrow c_i$$

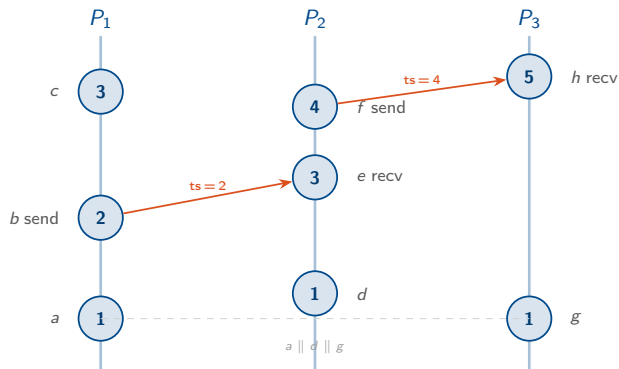
R3 — Ricezione messaggio

Prendi il massimo, poi incrementa:

$$c_j \leftarrow \max(c_j, m.ts) + 1$$

Il +1 in R3 è fondamentale: garantisce che $C(recv) > C(send)$, anche quando il mittente ha un clock molto più alto del destinatario.

Diagramma spazio-tempo con Lamport clock



Calcolo passo per passo:

- ▶ a, d, g : eventi interni $\rightarrow$ R1:
 $C = 1$ su tutti e tre.
- ▶ b : invio messaggio $\rightarrow$ R2: $C = 2$,
 $ts = 2$ allegato al messaggio.
- ▶ e : ricezione $\rightarrow$ R3:
 $\max(1, 2) + 1 = 3$.
- ▶ f : invio messaggio $\rightarrow$ R2: $C = 4$,
 $ts = 4$ allegato al messaggio.
- ▶ h : ricezione $\rightarrow$ R3:
 $\max(1, 4) + 1 = 5$.

Causalità rispettata:

$b \rightarrow e$: $C(b) = 2 < C(e) = 3 \checkmark$

$f \rightarrow h$: $C(f) = 4 < C(h) = 5 \checkmark$

Teorema — Clock Consistency

Le regole R1–R3 garantiscono: $a \rightarrow b \implies C(a) < C(b)$

Dimostrazione per i tre casi base di $\rightarrow$:

- ▶ *Ordine locale*: R1 incrementa c_i prima di ogni evento, quindi eventi successivi sullo stesso processo ricevono valori crescenti.
- ▶ *Passaggio di messaggi*: R2 garantisce $m.ts = C(\text{send})$; R3 garantisce $C(\text{recv}) = \max(\cdot) + 1 > m.ts$.
- ▶ *Transitività*: se $C(a) < C(b)$ e $C(b) < C(c)$, allora $C(a) < C(c)$ per transitività dell'ordine sui naturali.

Limite critico: l'implicazione inversa non vale

$C(a) < C(b)$ **non** implica $a \rightarrow b$.

Se $a \parallel b$, è possibile che $C(a) < C(b)$ per pura casualità numerica, senza alcun legame causale. Un osservatore esterno che vede solo i numeri non riesce a distinguere causalità da concorrenza. Per farlo serve una struttura dati più ricca: il *vector clock*.

Perché il Lamport clock non basta

Il Lamport clock risponde alla domanda: “se $a \rightarrow b$, il timestamp di a è minore di quello di b ?” — Sì.

Ma non risponde alla domanda inversa: “se il timestamp di a è minore di quello di b , allora $a \rightarrow b$?” — Non necessariamente.

Questo significa che guardando solo i numeri possiamo *ordinare* eventi (utile per snapshot e mutex), ma non possiamo dire se due eventi con timestamp $C(a) < C(b)$ siano davvero in relazione causale o semplicemente concorrenti.

Il **vector clock** risolve il problema codificando, in ogni timestamp, la *conoscenza parziale* che ogni processo ha del progresso degli altri.

Obiettivo

Costruire una struttura TS tale che:

$$a \rightarrow b \iff TS(a) < TS(b) \qquad a \parallel b \iff TS(a) \parallel TS(b)$$

Perché il Lamport clock non rileva la concorrenza

Rilevare la concorrenza significa rispondere a una domanda precisa: quando guardo due eventi a e b , riesco a capire se uno dei due *sapeva* dell'altro (aveva ricevuto informazioni dall'altro) quando è accaduto?

Con il Lamport clock

Vedo $C(a) = 3$ e $C(b) = 5$. So che $3 < 5$, ma non so il perché. Ci sono due spiegazioni possibili e il numero non le distingue:

- ▶ b è venuto dopo perché ha ricevuto un messaggio da a — relazione causale.
- ▶ b è venuto dopo perché il suo contatore era arrivato a 5 per fatti suoi, senza aver mai sentito parlare di a — concorrenza.

Verso i vector clock

Il Lamport clock conta *quante cose sono successe*. Serve, per ogni processo, dell'informazione sugli altri processi. Solo così possiamo distinguere causalità da concorrenza.

Vector clock: la struttura

Con il Lamport clock ogni processo tiene un solo numero. L'idea del Vector clock è semplice: invece di un numero, ogni processo tiene **un vettore di n numeri** — uno per ciascun processo nel sistema.

Ogni processo P_i mantiene $V_i \in \mathbb{N}^n$, inizialmente $[0, \dots, 0]$.

Cosa significa ogni cella

$V_i[k] =$ quanti eventi di P_k sono *noti* a P_i nel momento in cui avviene l'evento.

Esempio con tre processi:

Se P_2 ha vettore $[3, 2, 0]$ significa:

- ha visto 3 eventi di P_1 ,
- ha fatto 2 cose sue,
- non sa ancora nulla di P_3 .

È esattamente l'informazione che mancava al Lamport clock: non solo *quante* cose sono successe, ma *di chi* hai visto il lavoro.

Vector clock: le regole di aggiornamento

Le regole sono tre, proprio come nel Lamport clock, ma ora si aggiorna una sola cella del vettore — non tutto.

R1 — Evento interno:

Incrementa solo la tua cella.

$$V_i[i] \leftarrow V_i[i] + 1$$

R2 — Invio messaggio:

Incrementa la tua cella, poi allega l'intero vettore al messaggio.

$$V_i[i] \leftarrow V_i[i] + 1 \quad m.ts \leftarrow V_i$$

Il destinatario riceverà la tua “fotografia” del mondo in quel momento.

R3 — Ricezione messaggio:

Per ogni cella prendi il massimo tra il tuo valore e quello del mittente, poi incrementa la tua cella.

$$\forall k : V_j[k] \leftarrow \max(V_j[k], m.ts[k]) \quad V_j[j] \leftarrow V_j[j] + 1$$

Così assorbi tutto quello che il mittente sapeva, e aggiungi la ricezione come evento tuo.

Vector clock: confrontare due vettori

Dati due vettori V e W , la comparazione avviene **cella per cella**.

$V \leq W$ — V è minore o uguale a W :

ogni cella di V è minore o uguale alla corrispondente cella di W .

$$V \leq W \iff \forall k : V[k] \leq W[k]$$

$V < W$ — V è strettamente minore di W :

vale $V \leq W$ e almeno una cella è strettamente minore. Questo significa: chi ha V sapeva *meno* di chi ha W .

$V \parallel W$ — V e W sono incomparabili:

$V \not\leq W$ e $W \not\leq V$. Ci sono celle dove V è avanti e celle dove W è avanti — i due processi stavano lavorando nel buio uno rispetto all'altro.

Esempio:

$[2, 1, 0]$ vs $[1, 3, 0]$: nella prima cella il sinistro è avanti, nella seconda è indietro $\Rightarrow$ incomparabili $\Rightarrow$ **concorrenti**.

$[2, 1, 0]$ vs $[3, 2, 0]$: il sinistro è minore o uguale in ogni cella $\Rightarrow$ **relazione causale**.

Vector clock: il risultato fondamentale

Mettendo insieme la struttura e le regole di aggiornamento, si dimostra il teorema che giustifica tutto il meccanismo.

Teorema — Caratterizzazione completa (Fidge/Mattern, 1988)

$$a \rightarrow b \iff V(a) < V(b)$$

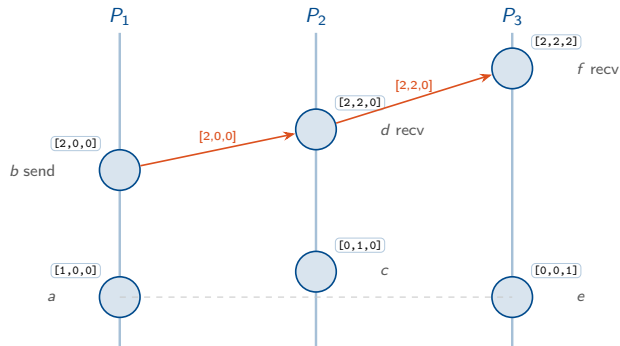
$$a \parallel b \iff V(a) \parallel V(b)$$

La freccia va in **entrambe le direzioni** — a differenza del Lamport clock dove valeva solo $a \rightarrow b \Rightarrow C(a) < C(b)$. Guardando i vettori puoi quindi stabilire con certezza se due eventi sono in relazione causale oppure concorrenti.

Qual è il costo?

Ogni messaggio trasporta n interi invece di 1. Per sistemi con molti processi questo può diventare oneroso. Non è però un difetto evitabile: si può dimostrare che non esiste nessuna struttura più compatta che caratterizzi $\rightarrow$ in modo completo. Il costo $O(n)$ per messaggio è il minimo possibile.

Diagramma spazio-tempo con Vector clock



Calcolo di d :

P_2 riceve $ts = [2,0,0]$

merge con $[0,1,0]$:

$\max(0, 2) = 2$, $\max(1, 0) = 1$,

$\max(0, 0) = 0$

poi +1 su P_2 : **[2,2,0]✓**

Calcolo di f :

P_3 riceve $ts = [2,2,0]$

merge con $[0,0,1]$:

$[2, 2, 1]$, poi +1 su P_3 : **[2,2,2]✓**

Concorrenza rilevata:

a : [1,0,0], c : [0,1,0]

$[1, 0, 0] \not\leq [0, 1, 0]$

$[0, 1, 0] \not\leq [1, 0, 0]$

$\Rightarrow a \parallel c$ ✓

Causalità confermata:

$b \rightarrow d$: $[2, 0, 0] < [2, 2, 0]$ ✓

$d \rightarrow f$: $[2, 2, 0] < [2, 2, 2]$ ✓

Il problema: una risorsa, molti contendenti

Immagina un file su un server che tre processi vogliono modificare. Se due processi lo modificano contemporaneamente, il risultato è corrotto — uno sovrascrive il lavoro dell'altro senza saperlo.

La **sezione critica** è la porzione di codice in cui un processo accede alla risorsa condivisa. Il problema della mutua esclusione è garantire che **al più un processo per volta** si trovi in quella porzione.

Definizione

Un protocollo di mutua esclusione distribuita deve garantire tre proprietà simultaneamente:

- ▶ **Safety:** due processi non entrano mai nella sezione critica nello stesso momento.
- ▶ **Liveness:** ogni processo che vuole entrare prima o poi ci riesce — niente attese infinite.
- ▶ **Fairness:** i processi entrano nell'ordine in cui hanno fatto richiesta — niente scavalcamenti.

Perché è difficile in un sistema distribuito

In un sistema con un solo processore il problema si risolve con un *lock*: una variabile in memoria condivisa che tutti possono leggere e scrivere istantaneamente.

In un sistema distribuito questa soluzione non esiste. I processi sono su macchine diverse, collegate solo da messaggi. Non c'è memoria condivisa, non c'è un orologio comune, e i messaggi ci mettono un tempo variabile ad arrivare.

La soluzione ovvia — e sbagliata

Si potrebbe eleggere un *coordinatore centrale* a cui tutti chiedono il permesso. Funziona, ma ha due difetti gravi:

- ▶ Se il coordinatore si guasta, il sistema si blocca — è un *single point of failure*.
- ▶ Tutti i messaggi passano per lui — è un *collo di bottiglia*.

Serve un protocollo **peer-to-peer**: i processi si coordinano tra loro, senza nessun coordinatore, usando solo messaggi diretti.

L'idea di Lamport: una coda condivisa senza memoria condivisa

Lamport (1978) osserva che il problema si riduce a costruire una **classifica delle richieste** che tutti i processi vedano nello stesso modo — senza memoria condivisa.

Ogni processo tiene una *coda locale* Q_i con tutte le richieste che conosce, ordinata per la coppia $\langle ts, i \rangle$: prima per timestamp Lamport, poi per ID processo come spareggio.

Perché lo spareggio sull'ID?

Se due richieste arrivano con lo stesso timestamp, senza uno spareggio non si sa chi passa prima. Usare l'ID del processo risolve il pareggio in modo deterministico: tutti i nodi applicano la stessa regola e costruiscono la stessa classifica identica.

Il risultato chiave

Anche se le richieste arrivano in ordine diverso sui vari nodi, l'ordinamento per $\langle ts, i \rangle$ garantisce che alla fine tutti abbiano la stessa coda — e quindi lo stesso accordo su chi deve entrare per primo.

Quando un processo può entrare nella sezione critica

Milano vuole entrare nella sezione critica. Manda una `REQUEST` a Roma e Napoli e poi aspetta. Per entrare devono essere vere **due condizioni insieme**.

Condizione 1 — essere in cima alla coda

La richiesta di Milano deve essere la prima nella sua coda locale: nessun altro ha una richiesta con timestamp più basso, o con lo stesso timestamp ma ID più basso.

Se Roma ha già una richiesta con timestamp minore, Milano aspetta.

Condizione 2 — nessuna richiesta in volo

Milano deve aver ricevuto almeno un messaggio recente di `ACK` da *ogni* altro processo — con timestamp maggiore del suo. Questo esclude che qualcuno abbia mandato una richiesta più prioritaria che è ancora in viaggio sulla rete.

Solo quando entrambe le condizioni sono vere

Milano sa con certezza che nessun altro ha una richiesta più prioritaria, né in coda né in transito. Solo allora accede alla risorsa condivisa.

I tre messaggi del protocollo

Il protocollo usa tre soli tipi di messaggio.

REQUEST(t, i) — voglio entrare

P_i incrementa il proprio clock, manda la REQUEST a tutti, e inserisce la propria richiesta nella coda locale Q_i . Poi aspetta.

ACK(t, i) — ho ricevuto la tua richiesta

Quando P_j riceve una REQUEST da P_i , aggiorna il proprio clock, inserisce la richiesta di P_i nella sua coda Q_j , e risponde con un ACK. L'ACK serve a Milano per sapere che Roma è aggiornata e non ha richieste in volo più vecchie.

RELEASE — ho finito, tocca al prossimo

Quando P_i esce dalla sezione critica, manda un RELEASE a tutti. Chi lo riceve rimuove la richiesta di P_i dalla propria coda e rivaluta se può entrare.

Costo: ogni ingresso e uscita dalla SC costa $3(n - 1)$ messaggi: $n-1$ REQUEST, $n-1$ ACK, $n-1$ RELEASE.

(1) Richieste simultanee

P_1 e P_2 inviano REQ con $ts = 1$ quasi nello stesso istante. P_3 non richiede la SC: risponde solo con ACK.

(2) ACK

Ogni nodo, ricevuta una REQ, inserisce la richiesta nella propria coda locale e invia immediatamente ACK al mittente.

(3) Ingresso di P_1

Coda di P_1 : $\langle 1, 1 \rangle, \langle 1, 2 \rangle$.

P_1 è in **cima** alla coda e ha ricevuto **2 ACK**: entra.

P_2 ha 2 ACK ma non è in cima: resta in attesa.

(4) Release

P_1 esce e invia RELEASE. P_2 rimuove $\langle 1, 1 \rangle$ dalla coda: ora è in cima con 2 ACK già ricevuti. Entra.